

Report

Documentation

Ilia Hajypour

Part 1 Task 1 report:

Ilia Hajypour Alvar / 250493135 /

Report

Part 1.1

Encapsulation

Encapsulation tells us that the object's data in my class is not directly accessible to the fields, though it allows us to have a controlled-by-method access. In the Typist class, all fields are declared private, which means they cannot be accessed or changed directly from outside the class. This makes us ensured that the typist's state always remains valid and consistent.

The private fields in the Typist class are: Name, Symbol, progress, burntOut, burnoutTurnsRemaining, accuracy.

Public methods access them for intersection.

Accessors and Mutators

Getters, so called as accessors just return the values, without changing them.

Those are: getName, getSymbol, getProgress, getAccuracy, getBurnoutTurnsRemaining, isBurntOut

These help the program to read the typist's current state safely.

On the other hand, setters or so called mutators which I have not heard of its name before, are modifier methods:

setAccuracy(double newAccuracy), setSymbol(char newSymbol), typeCharacter(), slideBack(int amount), burnOut(int turns), recoverFromBurnout(), resetToStart()

The Importance of Encapsulation

Encapsulation is important here because it allows validation rules to be applied. For example, `slideBack()` method ensures that progress never becomes negative by resetting it to 0 if needed. Similarly, `setAccuracy()` method ensures that accuracy always stays between 0.0 and 1.0 by clamping values outside that range. These checks matter because they keep the object in a valid state and ensure the typing race behaves correctly.

Validation in `slideBack` method

```
progress -= amount;
if (progress < 0) {
    progress = 0;
}
```

This validation matters because it prevents the typist from moving before the start of the race. Without this check, a mistype could make progress negative, which would violate the race rules and lead to incorrect behaviour in the program.

Validation in `setAccuracy` method

The `setAccuracy()` method ensures that accuracy always remains within the valid range of 0.0 to 1.0. It checks the new value before storing it:

- ❖ if the value is below 0.0, it stores 0.0
- ❖ if the value is above 1.0, it stores 1.0

❖ otherwise, it stores the value unchanged

Its logic is:

```
if (newAccuracy < 0.0) {  
    accuracy = 0.0;  
} else if (newAccuracy > 1.0) {  
    accuracy = 1.0;  
} else {  
    accuracy = newAccuracy;  
}
```

This validation matters because the assignment defines accuracy as a decimal value between 0 and 1. Regardless of having this protection, invalid values could be stored, which would break the program.

Report

Part 1.1.2

Testing:

To verify that the Typist class works correctly, I created a separate test class called Test. In this class, I created a Typist object and called its methods under different conditions. The console output was then used as evidence that each method behaved as expected.

Test 1: slideBack() does not allow progress below zero

What I tested:

I tested whether calling slideBack() could make progress shown negative.

How I tested this:

The typist started with progress 0. I then called slideBack(6).

Identified output:

Initial progress: 0

After slideBack(6): 0

Explanation:

This proves that the slideBack() method prevents progress from going negatively. Although 6 was subtracted, the method reset the value to 0 instead of allowing a negative result.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8117]
(c) Microsoft Corporation. All rights reserved.

C:\Users\BiaDigi.com>cd C:\Users\BiaDigi.com\OneDrive\Vscode\TypingRaceSimulator1\Part 1
C:\Users\BiaDigi.com\OneDrive\Vscode\TypingRaceSimulator1\Part 1>javac Typist.java TypistTest.java
C:\Users\BiaDigi.com\OneDrive\Vscode\TypingRaceSimulator1\Part 1>java TypistTest
~~~~~
TEST 1: slideBack() cannot go below 0
~~~~~
Initial progress: 0
After slideBack(6): 0
Expected result: progress should still be 0
```

Test 2: Burnout counts down correctly and resets at zero

What I tested:

I tested whether burnout decreases turn by turn and ends automatically when the remaining turns reach zero.

How I tested thia:

I called `burnOut(3)` on the typist and then called `recoverFromBurnout()` three times.

Identified output:

Burnout started: Burnt out = true, Turns remaining = 3

After 1st recovery: Burnt out = true, Turns remaining = 2

After 2nd recovery: Burnt out = true, Turns remaining = 1

After 3rd recovery: Burnt out = false, Turns remaining = 0

Explanation:

This test shows that the burnout system works correctly. Each call to `recoverFromBurnout()` reduces the remaining burnout turns by one. When the counter reaches 0, the typist is no longer burnt out. This confirms that burnout is temporary and clears correctly.

```
TEST 2: Burnout countdown works
Burnout started:
Burnt out? true
Turns remaining: 3

After 1st recovery:
Burnt out? true
Turns remaining: 2

After 2nd recovery:
Burnt out? true
Turns remaining: 1

After 3rd recovery:
Burnt out? false
Turns remaining: 0
Expected result: burnout cleared and turns = 0
```

Test 3: resetToStart method clears both progress and burnout state

What I tested:

I tested whether resetToStart() restores the typist to the starting state.

How I tested this:

I first moved the typist forward twice using typeCharacter(), then applied burnout for 2 turns using burnOut(2). After that, I called resetToStart().

Identified output:

Before reset:

Progress: 2

Burnt out: true

Burnout turns: 2

After reset:

Progress: 0

Burnt out: false

Burnout turns: 0

Explanation:

This proves that resetToStart() correctly clears both the typist's progress and burnout state. After resetting, the typist returns to the beginning of the race and any remaining burnout is removed.

```

=====
TEST 3: resetToStart() resets state
=====
Before reset:
Progress: 2
Burnt out: true
Burnout turns: 2

After reset:
Progress: 0
Burnt out: false
Burnout turns: 0
Expected result: progress = 0 and burnout cleared

```

Test 4: setAccuracy() keeps accuracy within the valid range 0.0 to 1.0

What I tested:

I tested whether setAccuracy() prevents invalid values from being stored.

How I tested this:

I called setAccuracy(1.75), setAccuracy(-0.13), and setAccuracy(0.65).

Identified output:

After setAccuracy(1.75): 1.0

After setAccuracy(-0.13): 0.0

After setAccuracy(0.65): 0.65

Explanation:

This test proves that setAccuracy() clamps values into the valid range. A value above 1.0 is reduced to 1.0, a value below 0.0 is increased to 0.0, and a valid value remains unchanged. This ensures that the accuracy field always stays within the allowed range.

```

#####
TEST 4: setAccuracy() clamps values
#####
After setAccuracy(1.75): 1.0
After setAccuracy(-0.13): 0.0
After setAccuracy(0.65): 0.65
Expected result: values limited between 0.0 and 1.0

```

Test 5: typeCharacter method moves the typist forward

What I tested:

I tested whether typeCharacter() increases progress correctly during normal typing.

How I tested this:

I reset the typist to the start and then called typeCharacter() twice.

Identified output:

Initial progress: 0

After 1st typeCharacter(): 1

After 2nd typeCharacter(): 2

Explanation:

This test confirms that typeCharacter() moves the typist forward by one character each time it is called, as long as the typist is not burnt out. The output shows normal forward movement working correctly.

```
~~~~~  
TEST 5: typeCharacter() moves forward  
~~~~~  
Initial progress: 0  
After 1st typeCharacter(): 1  
After 2nd typeCharacter(): 2  
Expected result: progress increases normally
```

Part 1 Task 2 report:

PROBLEM NUMBER 1: THERE IS NO RESET FOR THE THIRD TYPIST AT THE BEGINNING

PROBLEM DESCRIPTION:

In the previous version of the (in the file sent before) startRace() method, only seat1Typist and seat2Typist are reset before the race happens:

```
public void startRace()
{
    boolean finished = false;

    // Reset all typists to the start of the passage
    // (Ty was in a hurry here)
    seat1Typist.resetToStart();
    seat2Typist.resetToStart();
}
```

Where is the third typist?!

WHAT WILL HAPPEN GIVEN THIS PROBLEM:

This causes unfair behavior in the race. If the third typist has leftover progress or burnout state from a previous race, they would not start from the correct initial state. As a result, the race would not be fair because one typist may begin with an advantage or with an incorrect burnout condition.

For example, if seat3Typist had already progressed several characters in an earlier race and was not reset, they could begin the next race ahead of the others.

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should reset all three typists before the race starts, so that every competitor begins from:

progress = 0

not burnt out

burnout turns remaining = 0

WHAT HAS BEEN FIXED THEN:

In the corrected version, I added the missing reset call:

```
boolean finished = false;
Typist winner = null;

// Reset all typists to the start of the passage
seat1Typist.resetToStart();
seat2Typist.resetToStart();
seat3Typist.resetToStart();
```

WHY:

This ensures that all three typists begin the race in the same starting condition. It removes any leftover state from earlier races and makes the race fair and consistent.

PROBLEM NUMBER 2: THE RACE COULD CRASH IF ANY EMPTY SEATS IS FOUND

PROBLEM DESCRIPTION:

In the previous version of the (in the file sent before) TypingRace code, the program assumes that all three seats already contain valid Typist objects. For example, in startRace() it immediately calls:

```
public void startRace()
{
    boolean finished = false;

    // Reset all typists to the start of the passage
    // (Ty was in a hurry here)
    seat1Typist.resetToStart();
    seat2Typist.resetToStart();
```

And later:

```
private void printSeat(Typist theTypist)
{
    int spacesBefore = theTypist.getProgress();
    int spacesAfter = passageLength - theTypist.getProgress();
```

Also, methods like raceFinishedBy() and printSeat() directly call methods on the typist object without first checking whether it is null.

WHAT WILL HAPPEN GIVEN THIS PROBLEM:

This can cause a crash in the form of a NullPointerException.

For example, if one of the seats has not been filled and still contains null, then a line such as:

```
seat1Typist.resetToStart();
```

or :

```
theTypist.getProgress();
```

will fail because the program is trying to call a method on an object that does not exist.

So instead of running the race, the program may terminate unexpectedly.

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should make sure that the race only starts when all three seats are filled, or it should safely handle missing typists.

At minimum, the program should check whether any seat is null before trying to use it. If a seat is empty, the race should not start and the user should be informed.

WHAT HAS BEEN FIXED THEN:

In the corrected version, I added a check at the start of `startRace()`:

```
public void startRace()
{
    if (seat1Typist == null || seat2Typist == null || seat3Typist == null)
    {
        System.out.println("Cannot start race: all three seats must be filled.");
        return;
    }
}
```

I also added defensive null checks in helper methods such as `raceFinishedBy()`, `printSeat()` and `advanceTypist()`.

WHY:

This fix prevents the program from attempting to use a missing typist object. Instead of crashing, the program now behaves safely and clearly tells the user what is wrong.

This makes the race simulation more robust and ensures that the program only runs when it has all required competitors.

PROBLEM NUMBER 3: THE WIN CONDITION IN METHOD RACEFINISHEDBY() IS INCORRECT

PROBLEM DESCRIPTION:

In the downloaded version of the code, the raceFinishedBy() method checks whether a typist's progress is exactly equal to the passage length:

```
private boolean raceFinishedBy(Typist theTypist)
{
    // Ty was confident this condition was correct
    if (theTypist.getProgress() == passageLength)
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

This means the program only declares a typist the winner if their progress lands on the passage length exactly.

WHAT WILL HAPPEN GIVEN THIS PROBLEM:

This causes wrong output / incorrect race behavior.

According to the specification, a typist should win as soon as their progress reaches or exceeds the passage length. If a typist overshoots the passage length, the original code may fail to recognize them as the winner.

For example, if the passage length is 20 and a typist's progress becomes 21, they have clearly completed the race. However, the original code would return false because $21 == 20$ is not true.

This means the race may continue even though a typist has already finished.

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should declare a typist the winner when their progress is greater than or equal to the passage length.

In other words, the condition should be:

`theTypist.getProgress() >= passageLength`

This handles both:

- finishing exactly on the last character

- overshooting the finish line

WHAT HAS BEEN FIXED THEN:

In the corrected version, I replaced the old condition with:

```
private boolean raceFinishedBy(Typist theTypist)
{
    if (theTypist == null)
    {
        return false;
    }

    return theTypist.getProgress() >= passageLength;
}
```

WHY:

This fix matches the assignment requirements because the race should end as soon as a typist has completed the full passage, even if their progress goes beyond the target length.

Using `>=` ensures that the winner is always recognized correctly and prevents the race from continuing after someone has already crossed the finish line.

PROBLEM NUMBER 4: THE MISTYPE PROBABILITY LOGIC IS WRITTEN CONVERSELY

PROBLEM DESCRIPTION:

In the original `advanceTypist()` method, the mistype check is written as:

```
// Mistype check – the probability should reflect the typist's accuracy
if (Math.random() < theTypist.getAccuracy() * MISTYPE_BASE_CHANCE)
{
    theTypist.slideBack(SLIDE_BACK_AMOUNT);
}
```

However, the specification says that the chance of a mistype should **decrease** for more accurate typists.

WHAT WILL HAPPEN GIVEN THIS PROBLEM:

This causes **wrong** behavior in the race simulation.

The original formula makes typists with higher accuracy more likely to mistype, which is the opposite of what the assignment requires.

For example, if `MISTYPE_BASE_CHANCE = 0.3`:

a typist with accuracy 0.9 gets mistype chance $0.9 \times 0.3 = 0.27$

a typist with accuracy 0.4 gets mistype chance $0.4 \times 0.3 = 0.12$

This means the more accurate typist is actually more likely to mistype, which is unfair and incorrect

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should make mistypes less likely when accuracy is higher and more likely when accuracy is lower.

So the probability should be based on the typist's inaccuracy, not their accuracy. A suitable formula is:

`(1.0 - theTypist.getAccuracy()) * MISTYPE_BASE_CHANCE`

This way:

high accuracy → low mistype chance

low accuracy → high mistype chance

WHAT HAS BEEN FIXED THEN:

In the corrected version, I changed the mistype logic to:

```
// More accurate typists should be LESS likely to mistype
if (Math.random() < (1.0 - theTypist.getAccuracy()) * MISTYPE_BASE_CHANCE)
{
    theTypist.slideBack(SLIDE_BACK_AMOUNT);
    justMistyped = true;
}
```

WHY:

This fix matches the intended race mechanics described in the specification. More accurate typists are now less likely to mistype, while less accurate typists are more likely to slide backwards.

This makes the race fairer and ensures that the behavior of the simulation reflects the typists' accuracy values properly.

PROBLEM NUMBER 5: THE WINNER IS NOT DISPLAYED WHEN THE RACE ENDS

PROBLEM DESCRIPTION:

In the original startRace() method, the race stops when a typist finishes, but the program does not print the winner's name. At the end of the method, there is only a comment:

```
// TODO (Task 2a): Print the winner's name here
```

This shows that the winner announcement was left unfinished.

WHAT WILL HAPPEN GIVEN THIS PROBLEM:

This causes missing output.

Although the race simulation runs and eventually ends, the user is not told who won. This makes the result unclear and means the program does not fully satisfy the task requirement, which specifically asks for the name of the winning typist to be displayed in the terminal once the race concludes.

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should:

- 1-identify which typist finished first
- 2-store that typist as the winner
- 3-print the winner's name after the race loop ends

This should happen automatically once the race is complete.

WHAT HAS BEEN FIXED THEN:

In the corrected version, I introduced a winner variable inside startRace():

```
Typist winner = null;  
double winnerOldAccuracy = 0.0;
```

Then, when checking whether the race has finished, I store the correct typist:

```
// Check if any typist has finished the passage  
if (raceFinishedBy(seat1Typist))  
{  
    finished = true;  
    winner = seat1Typist;  
}  
else if (raceFinishedBy(seat2Typist))  
{  
    finished = true;  
    winner = seat2Typist;  
}  
else if (raceFinishedBy(seat3Typist))  
{  
    finished = true;  
    winner = seat3Typist;  
}
```

Finally, after the race ends, I print the result:

```
if (winner != null)  
{  
    System.out.println("And the winner is... " + winner.getName() + "!");  
    System.out.println("Final accuracy: "  
        + format2dp(winner.getAccuracy())  
        + " (improved from "  
        + format2dp(winnerOldAccuracy)  
        + ")");  
}
```

WHY:

This fix ensures that the program clearly announces the winner at the end of the race, which is exactly what the task requires.

It also improves usability, because the user no longer has to guess who won by looking at the final race state. The program now gives a clear and direct result in the terminal.

PROBLEM NUMBER 6: RACE DISPLAY MAY BREAK WHEN A TYPIST OVERSHOOTS THE PASSAGE LENGTH

PROBLEM DESCRIPTION:

In the original `printSeat()` method, the spacing after the typist's symbol is calculated using:
`int spacesAfter = passageLength - theTypist.getProgress();`

This calculation assumes that the typist's progress will always be less than or equal to the passage length.

WHAT WILL HAPPEN GIVEN THIS PROBLEM

This can cause **incorrect output and broken display formatting**.

If a typist's progress becomes greater than the passage length (for example, if they type past the final character), the value of `spacesAfter` becomes negative. When the program then tries to print the race lane, the spacing will be incorrect and the visual layout of the race can break.

For example:

Passage length = 20

Typist progress = 22

Then:

`spacesAfter = 20 - 22 = -2`

Negative spacing does not make sense in the race display and can distort the output.

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should ensure that the typist's printed position never exceeds the passage length. If the typist's progress is greater than the passage length, the display should treat it as if the typist has reached the finish line.

This can be done by limiting the printed progress to the maximum passage length.

WHAT HAS BEEN FIXED THEN:

In the corrected version, I introduced a variable that caps the printed progress value:

```
int printedProgress = theTypist.getProgress();
if (printedProgress > passageLength)
{
    printedProgress = passageLength;
}
```

Then the spacing values are calculated using this safe value:

```
int spacesBefore = printedProgress;
int spacesAfter = passageLength - printedProgress;
```

WHY:

This fix ensures that the race display remains visually correct even if a typist's progress goes beyond the passage length. The typist is simply shown at the finish line instead of exceeding the display boundary.

As a result, the race output remains properly formatted and the simulation behaves consistently.

PROBLEM NUMBER 7: BURNOUT LEGEND DOES NOT MATCH THE SYMBOL DISPLAYED IN THE RACE

PROBLEM DESCRIPTION:

In the original printRace() method, the legend printed at the bottom of the race display states:

```
System.out.println(" [zz] = burnt out [<] = just mistyped");
```

However, in the printSeat() method, when a typist is burnt out the program actually prints the symbol ~ after the typist's character:

```
if (theTypist.isBurntOut())
{
    System.out.print('~');
    spacesAfter--; // symbol + ~ together take two characters
}
```

This means the symbol shown in the race lane does not match the symbol described in the legend.

WHAT WILL HAPPEN GIVEN THIS PROBLEM:

This causes confusing output for the user.

The legend suggests that [zz] represents a burnt-out typist, but the race display actually shows ~. As a result, users may not understand what the burnout symbol means when watching the race simulation.

WHAT IS THE RIGHT ACTION TO DO:

A correct implementation should ensure that the legend and the actual race display use the same symbol to represent burnout. The program should either:

- update the legend to match the symbol being printed, or

- change the displayed burnout symbol so that it matches the legend.

Consistency between the legend and the displayed symbols is important so that the user can easily understand the race status.

WHAT HAS BEEN FIXED THEN:

In the corrected version, the legend was updated so that it matches the burnout symbol printed in the race:

```
System.out.println(" [~] = burnt out  [<] = just mistyped");
```

This aligns the legend with the symbol printed in `printSeat()`.

WHY:

This fix ensures that the legend accurately describes the symbols shown in the race display. As a result, users can clearly interpret when a typist is burnt out, improving the clarity and usability of the race simulation.

The

END

•